3D Vision Camera Prototype

Requirements

1 source & 3 sinks

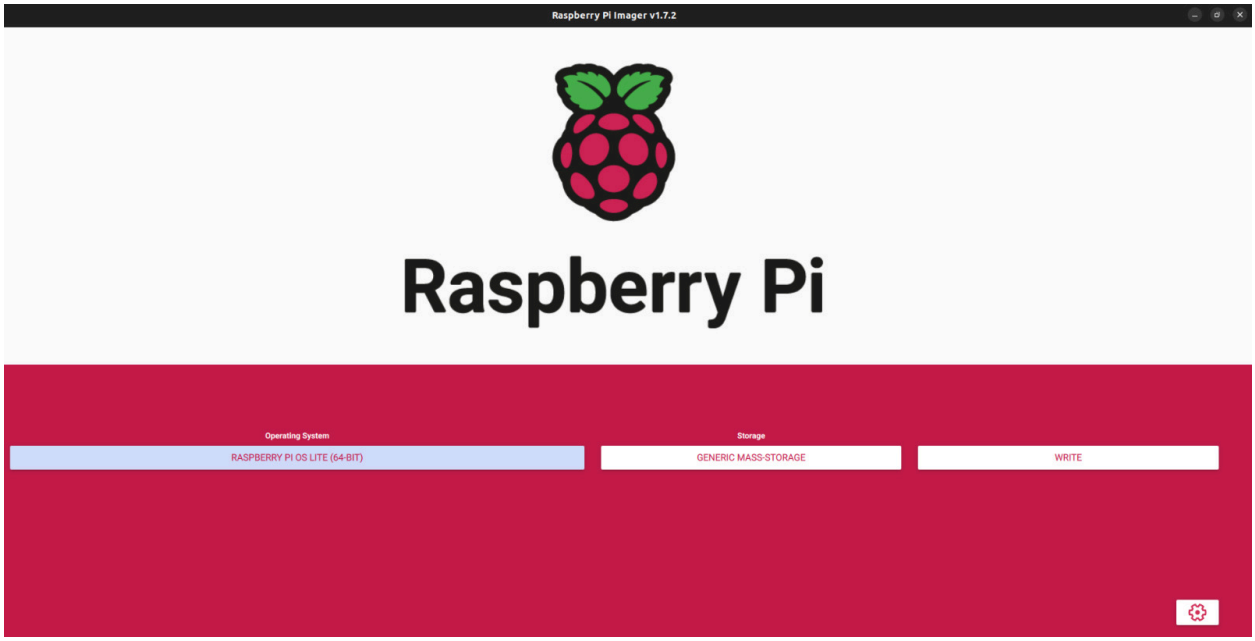
Sl No	Component	Quantity
1	GS Camera Mount (with soldering)	4
2	Camera Lens	4
3	Raspberry Pi 4 B (with power adapter)	4
4	SD cards 64 GB	4
5	Tall Tripod stands	4
6	Ball bearing	4
7	Tiny breadboards	4
8	10k ohm resistors	2
9	Single potential line connectors	3
8	LAN Cables	5
9	WiFi Router	1
10	Extension boxes	5 minimum, more based on required length
11	Three wired crimped connectors	3

Raspberry Pi Setup

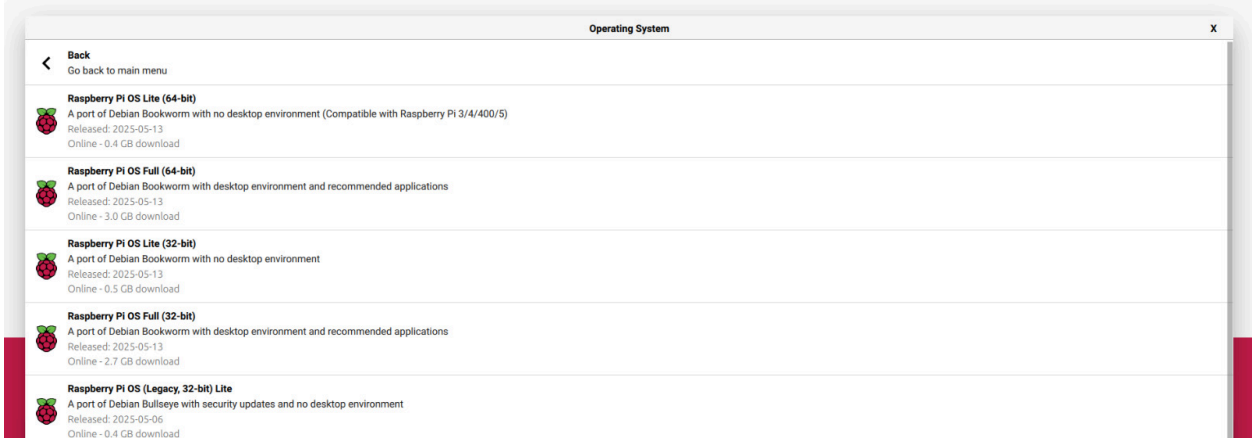
1. Flash Raspberry Pi OS (Bookworm Lite)

Requirements and steps:

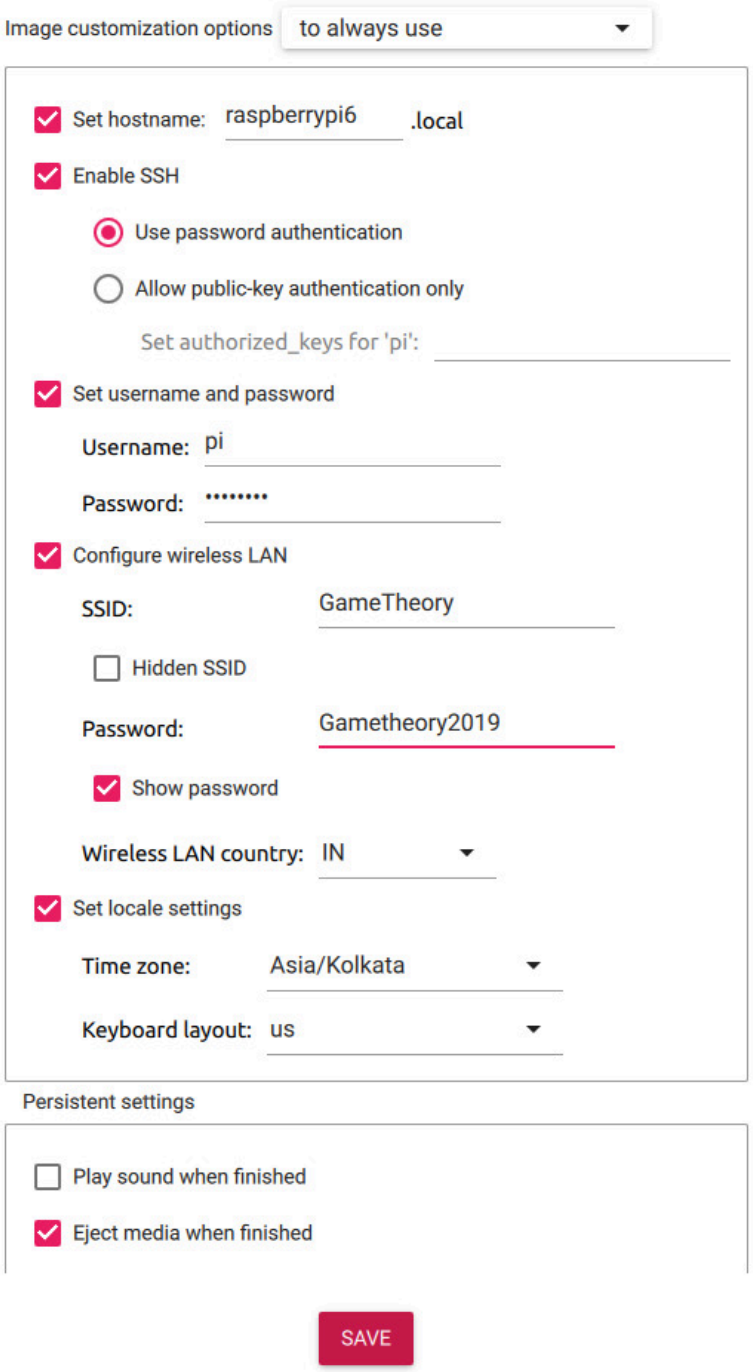
- Raspberry Pi Imager (or dd / BalenaEtcher)



- MicroSD card (min. 16GB recommended)
- Select OS: Raspberry Pi OS Lite (64-BIT Bookworm) .img file and also select the sd card as the storage.



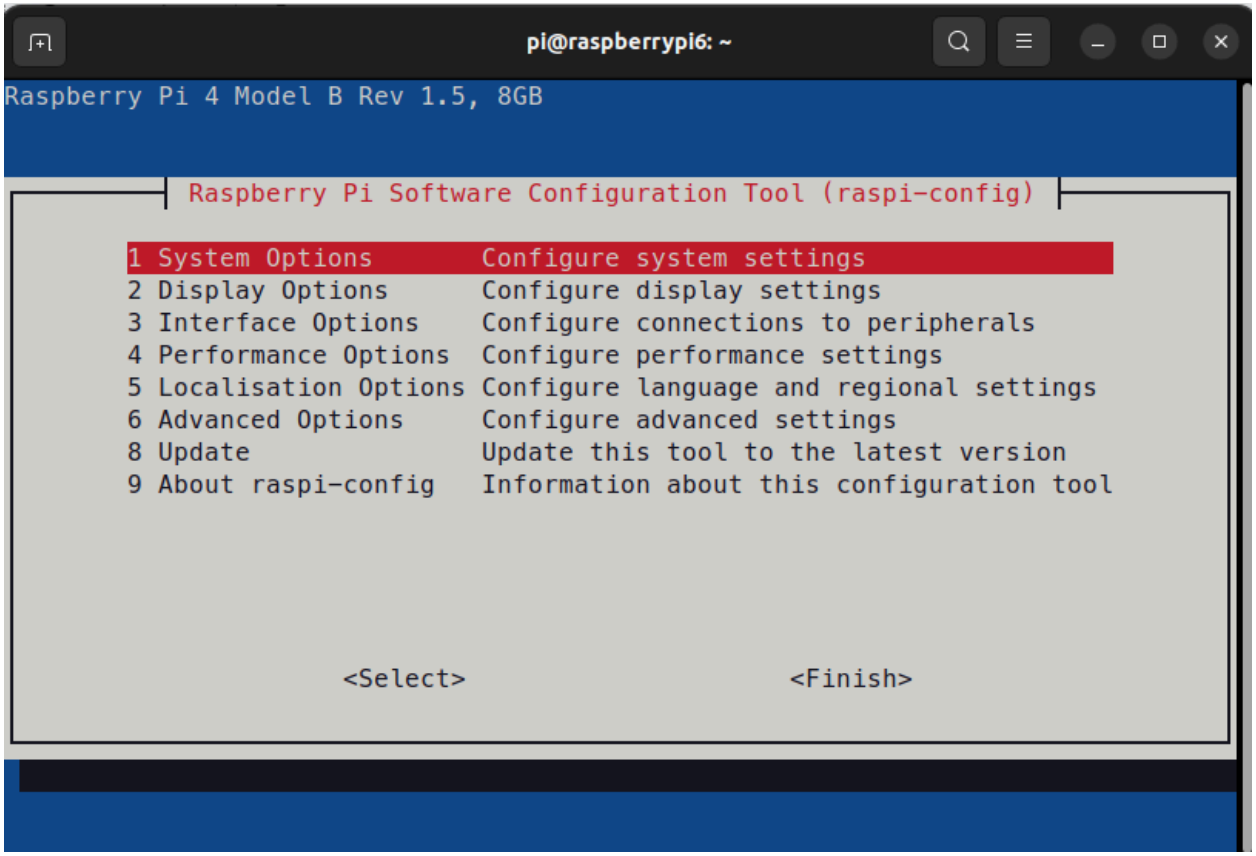
- Click on the settings icon (bottom right) and do the network configuration as shown in the below image then save and click on write.



2. Initial SSH Setup

Insert the sd card into pi4, the network configuration will help if you are connecting the pi via lan. For wifi setup:

1. Ssh into the pi. (You need to connect to lan so that you can ssh into the pi)
2. Type sudo raspi-config. This will open an interactive window.



3. Go to System Options -> Wireless LAN, then enter username and password of the wifi you want to connect to.

Once booted: (can skip this if already have done proper conf during flashing the sd card)

1. Connect Pi to monitor + keyboard (or via SSH if headless enabled).

2. Login (default user: `pi`, password: `qwaszx99`).
3. Set hostname:

```
sudo hostnamectl set-hostname raspberrypi11
```

4. Change password:

```
passwd
```

5. Ensure SSH is running:

```
sudo systemctl enable ssh
sudo systemctl start ssh
```

3. Ethernet Networking Options

You can connect Raspberry Pis using Ethernet via:

Option A: Single Router Setup

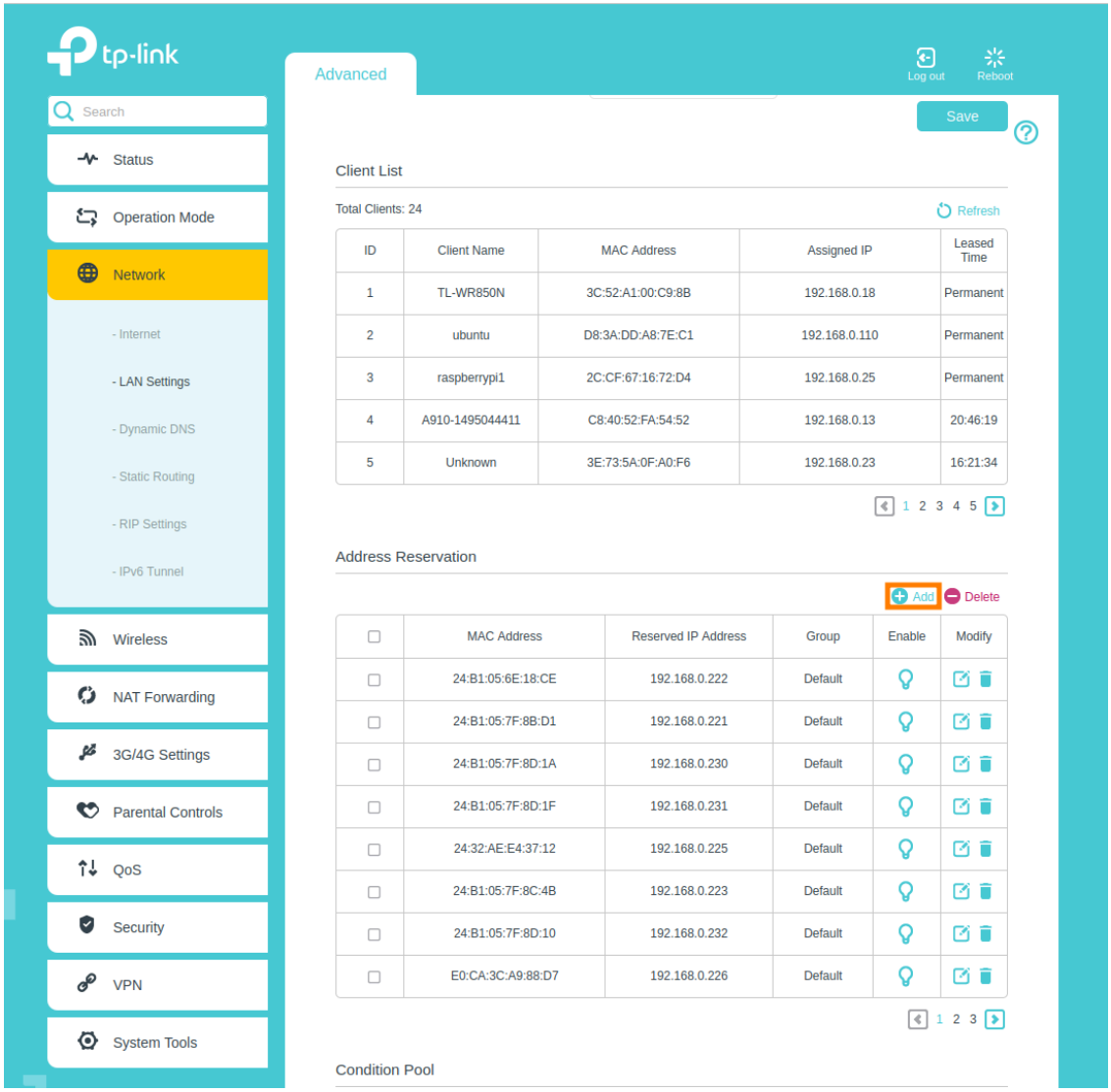
Steps

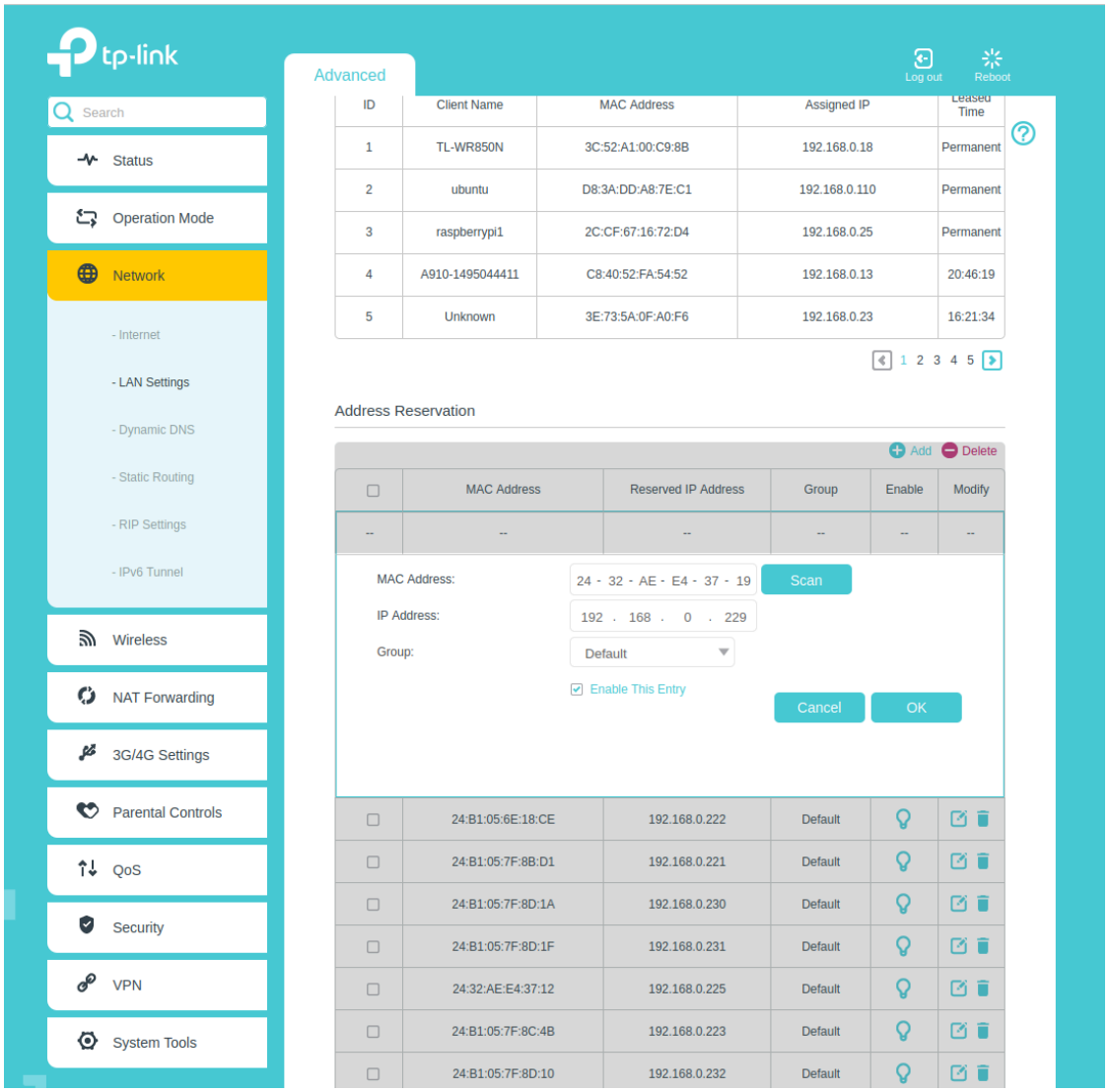
1. Connect the Pi via Ethernet to your router.
2. Access the router's admin page (usually 192.168.0.1 or 192.168.1.1).
3. Get the pi’s local IP and MAC address using the following command.

```
ifconfig -a
```

```
(base) ankith-varun@gt-thinkpad:~$ ifconfig -a
enp0s31f6: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether c4:c6:e6:fa:3d:97  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 19  memory 0xaad00000-aad20000

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 135559  bytes 32266244 (32.2 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 135559  bytes 32266244 (32.2 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.100  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::6aae:6e4b:212c:c48e  prefixlen 64  scopeid 0x20<link>
        ether 28:a0:6b:09:2a:68  txqueuelen 1000  (Ethernet)
        RX packets 1305544  bytes 696850114 (696.8 MB)
        RX errors 0  dropped 452  overruns 0  frame 0
        TX packets 310389  bytes 130414685 (130.4 MB)
        TX errors 0  dropped 91 overruns 0  carrier 0  collisions 0
```

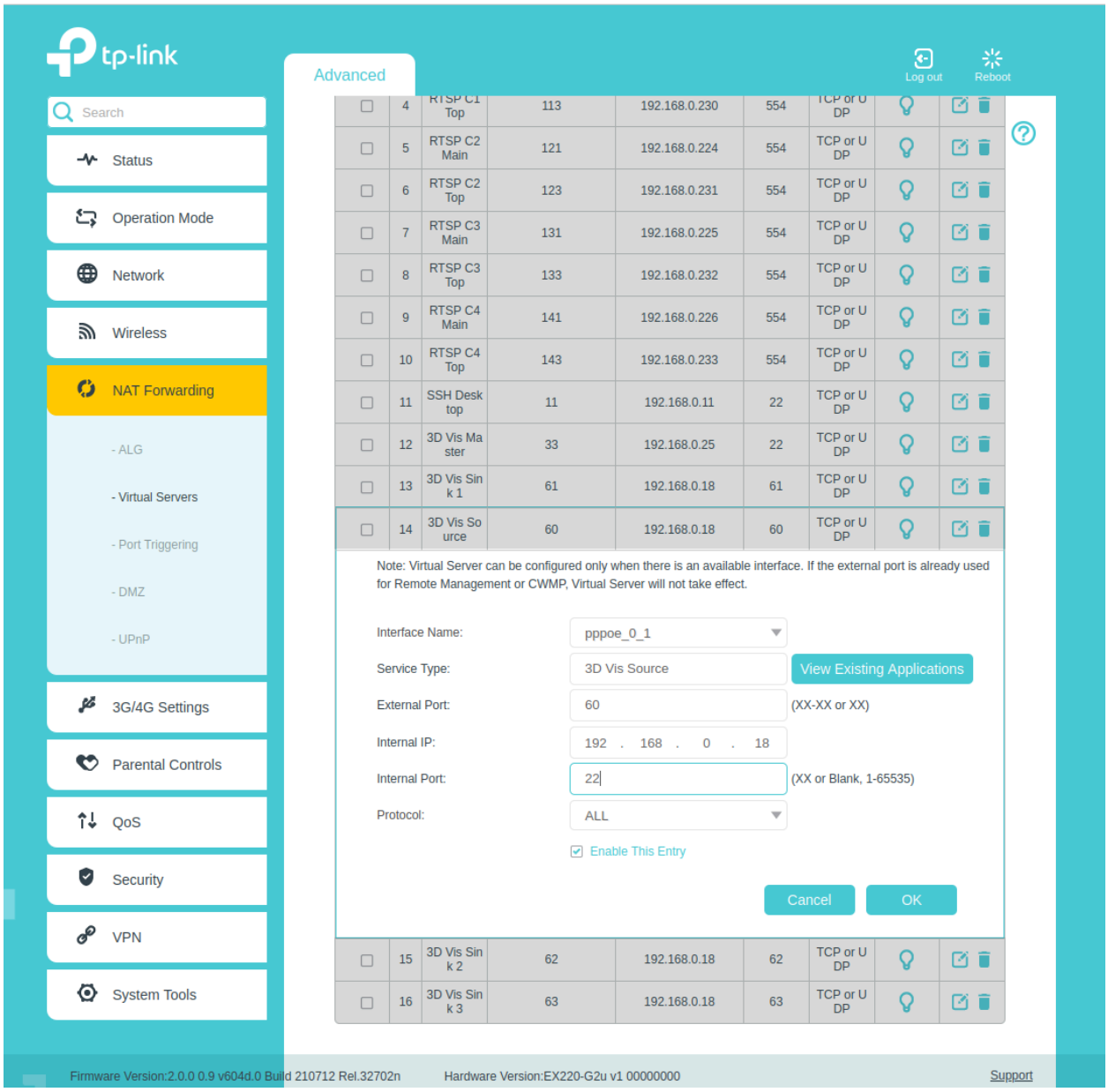
In the router admin page, head to Network -> LAN Settings and reserve the required IP to the MAC address of the pi.


tp-link
Advanced
Log out
Reboot
Search
Save
Status
Operation Mode
Network
- Internet
- LAN Settings
- Dynamic DNS
- Static Routing
- RIP Settings
- IPv6 Tunnel
Wireless
NAT Forwarding
3G/4G Settings
Parental Controls
QoS
Security
VPN
System Tools
Client List
Total Clients: 24
Refresh
ID | Client Name | MAC Address | Assigned IP | Leased Time
1 | TL-WR850N | 3C:52:A1:00:C9:8B | 192.168.0.18 | Permanent
2 | ubuntu | D8:3A:DD:A8:7E:C1 | 192.168.0.110 | Permanent
3 | raspberrypi1 | 2C:CF:67:16:72:D4 | 192.168.0.25 | Permanent
4 | A910-1495044411 | C8:40:52:FA:54:52 | 192.168.0.13 | 20:46:19
5 | Unknown | 3E:73:5A:0F:A0:F6 | 192.168.0.23 | 16:21:34
1 2 3 4 5
Address Reservation
Add
Delete
MAC Address | Reserved IP Address | Group | Enable | Modify
24:B1:05:6E:18:CE | 192.168.0.222 | Default
24:B1:05:7F:8B:D1 | 192.168.0.221 | Default
24:B1:05:7F:8D:1A | 192.168.0.230 | Default
24:B1:05:7F:8D:1F | 192.168.0.231 | Default
24:32:AE:E4:37:12 | 192.168.0.225 | Default
24:B1:05:7F:8C:4B | 192.168.0.223 | Default
24:B1:05:7F:8D:10 | 192.168.0.232 | Default
E0:CA:3C:A9:88:D7 | 192.168.0.226 | Default
1 2 3
Condition Pool



4. Setup port forwarding: Go to NAT Forwarding -> Virtual Servers and add an entry. For example,
    - Service Type: 3D Vis Source
    - External Port: e.g., 60
    - Internal IP: 192.168.x.x (your Pi's IP)
    - Internal Port: 22 (SSH)
    - Protocol: ALL

Repeat for each Pi, mapping different external ports.



Option B: Multi-Router (Cascaded NAT) Setup

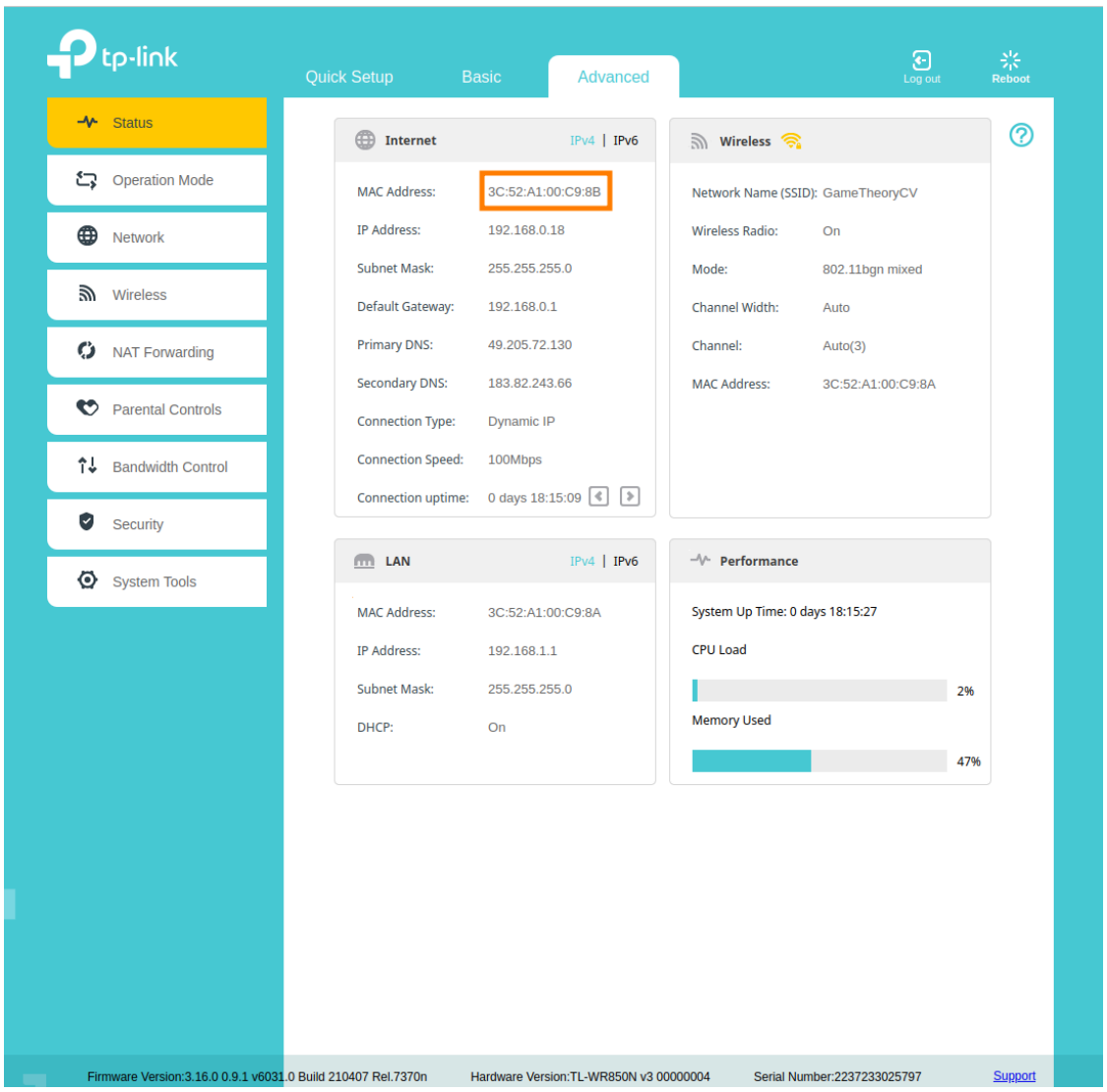
Topology

Parent Router ⇄ Child Router ⇄ Raspberry Pi

Steps

1. Connect Child Router WAN port to LAN port of Parent Router and the pi to LAN port of Child Router.

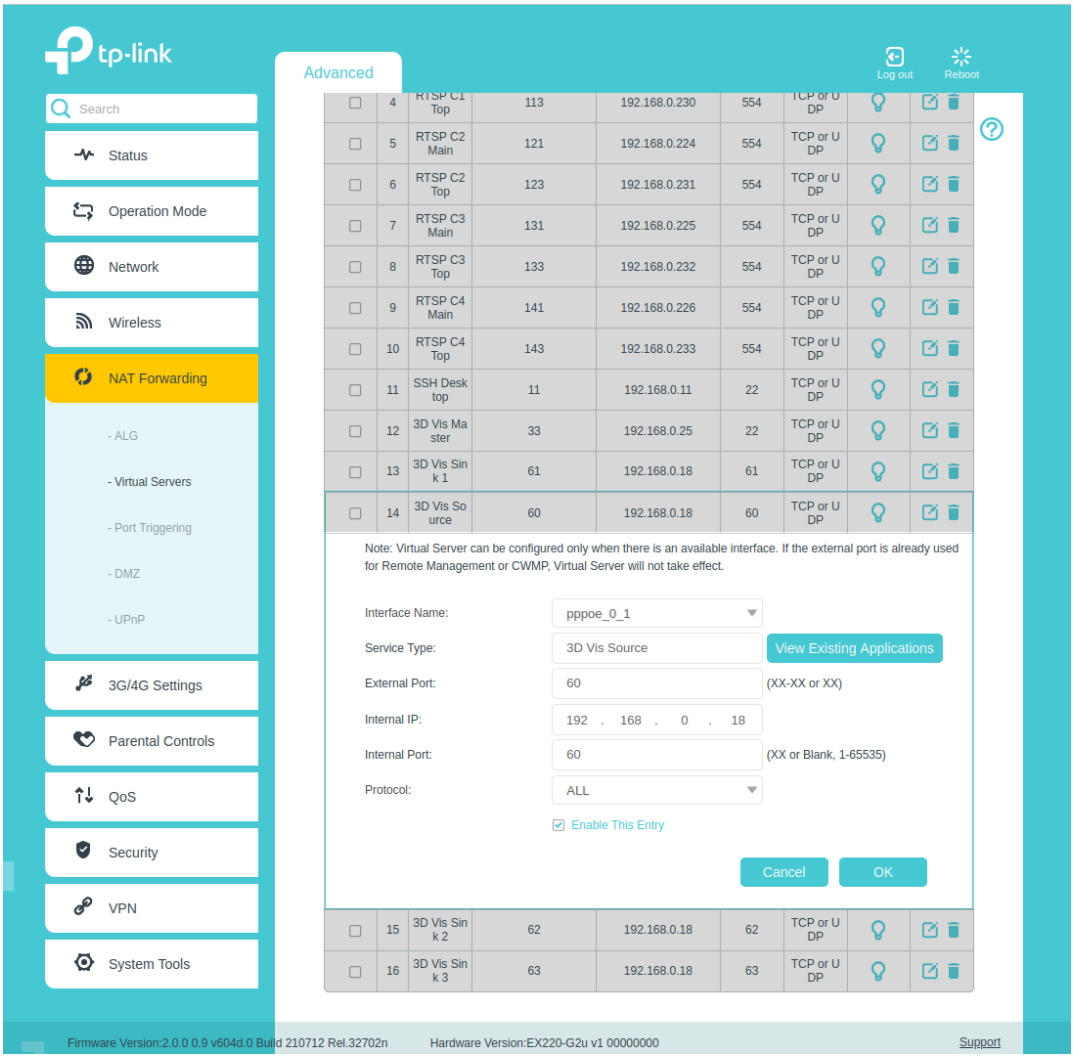
2. Get the MAC address of Child Router from its admin page as shown below.



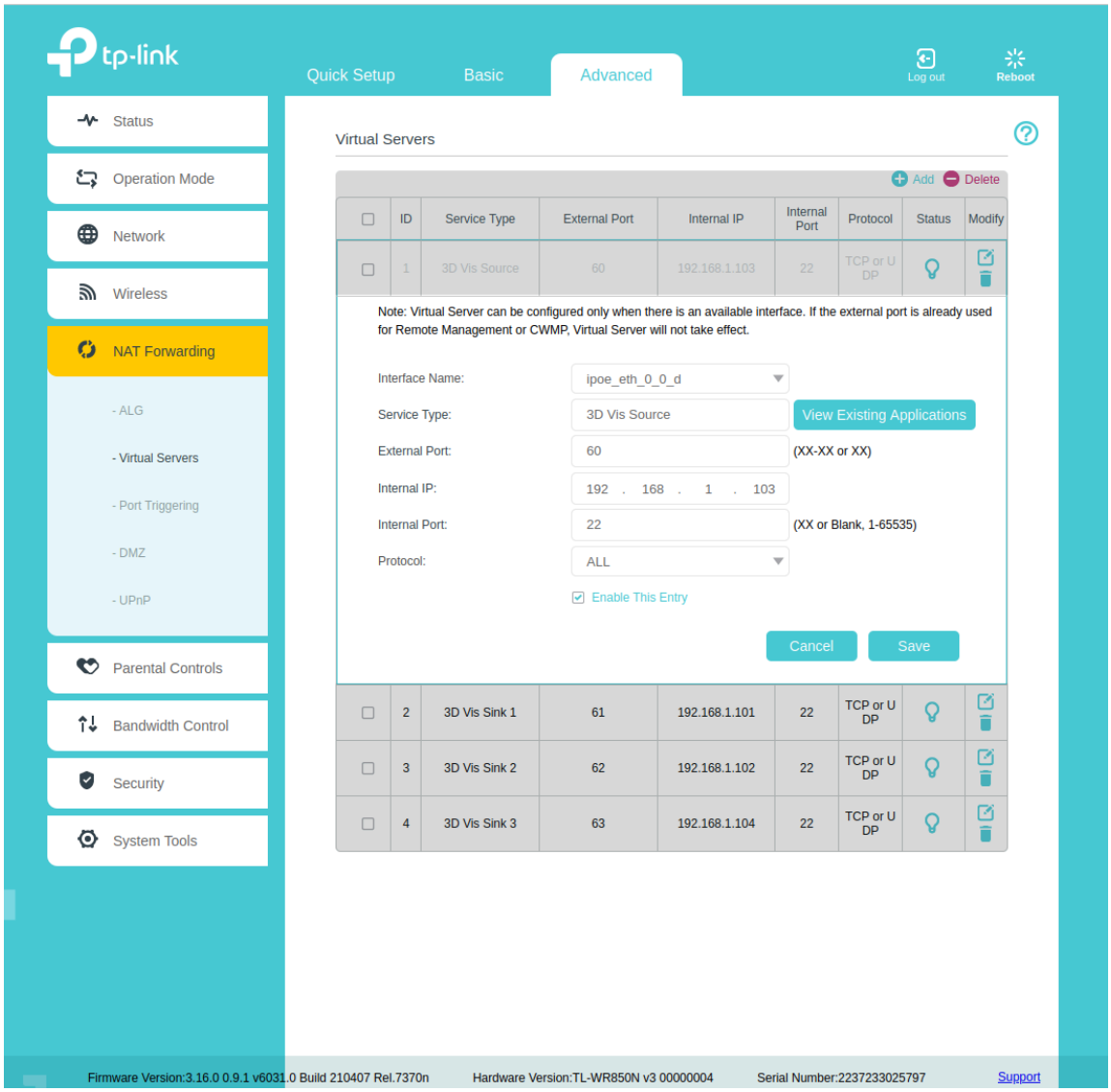
Assign static IP (e.g., 192.168.0.18) to the child router on the parent router admin page as explained in step 3A.3.

3. On Parent Router:

   - Forward port (e.g., 60) to Child Router IP:60 on NAT Forwarding -> Virtual Servers.



4. On Child Router:
   - Get the MAC address of the pi and reserve an IP as explained in step 3A.3.
   - Forward port 60 to Raspberry Pi IP:22 on NAT Forwarding -> Virtual Servers .



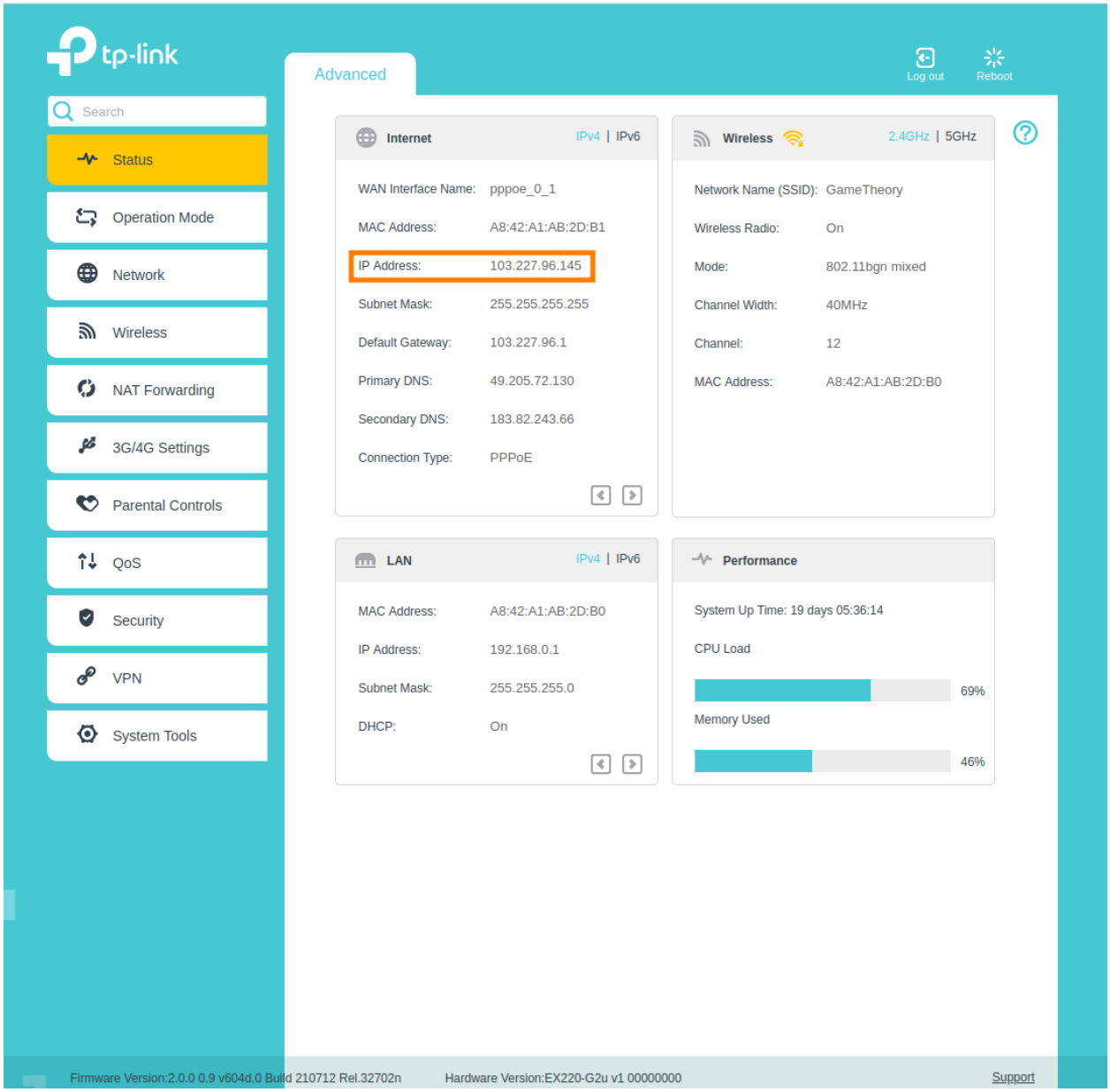
Repeat cascading setup for each Pi with unique port mappings.

---

4. Global SSH Access Test

To test:

1. Get the global IP address of Parent Router from its admin page.



2. From a remote machine:

```
ssh pi@<global-ip> -p <external-port>
```

If successful, you have SSH access over the internet.

Library Setup

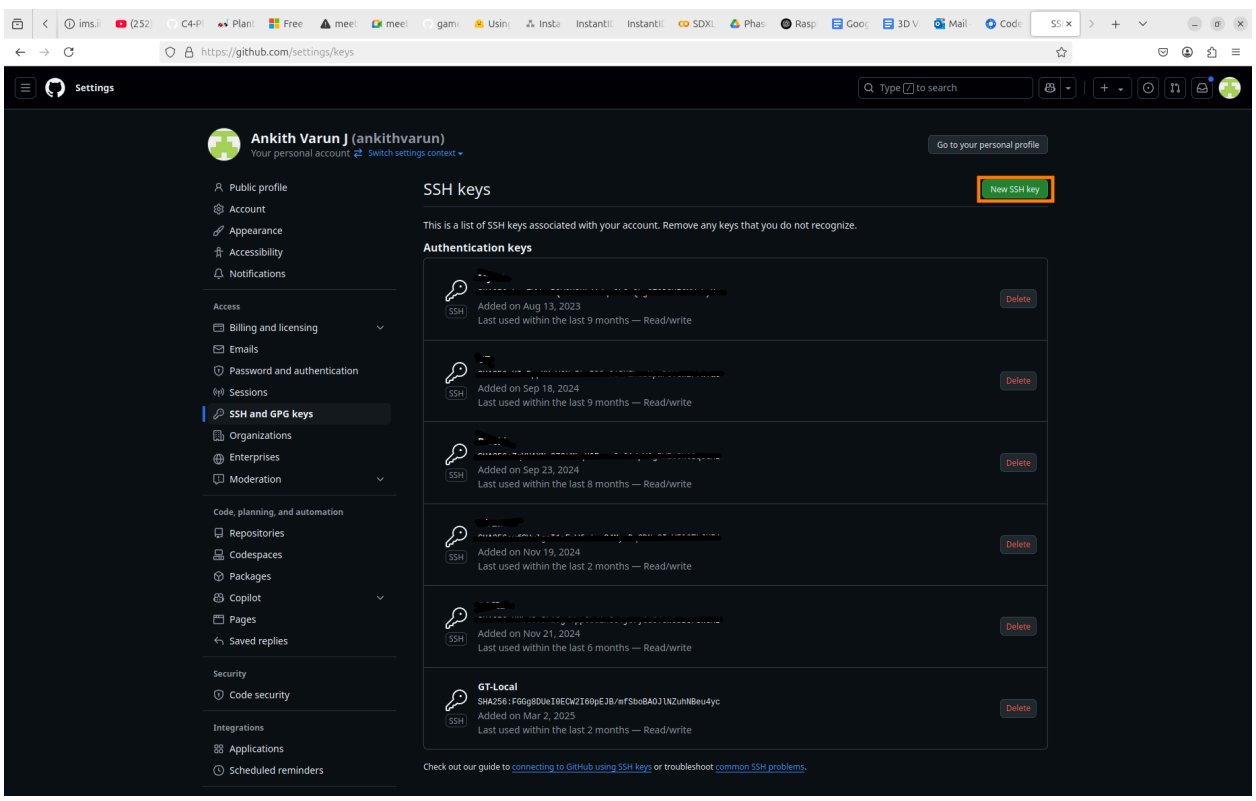
1. Github setup

a. Configure your github account using your github key. Run the following commands on the terminal of the pi.

```
ssh-keygen -t ed25519 -C "your_email@example.com"
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

Copy the contents.

b. Go to https://github.com/settings/keys and click "**New SSH Key**".



c. Paste the copied key in step 1 into the "**Key**" field and give it a title (e.g., `Raspberry Pi`) and click "**Add SSH key**"

d. Now install git, clone the repo and rename it as sink/source accordingly.

```
sudo apt install -y git
git clone git@github.com:gametheoryresearch/rpicam-apps.git {source/sink}_code
```

```
cd {source/sink)_code
git checkout new_{source/sink}
```

2. `LibCamera` setup

a.First step is to go in rpicam-apps folder(cloned from github).
b.`cd libcamera`
`git checkout 29156679`

```
meson setup build --buildtype=release -Dpipelines=rpi/vc4,rpi/pisp
-Dipas=rpi/vc4,rpi/pisp -Dv4l2=true -Dgstreamer=enabled -Dtest=false
-Dlc-compliance=disabled -Dcam=disabled -Dqcam=disabled -Ddocumentation=disabled
-Dpycamera=enabled
```

`ninja -C build`

```
sudo ninja -C build install
```

3. Camera libraries setup

a. Preliminaries

Install necessary libraries.

```
sudo apt-get update
sudo apt install -y python3-pip python3-picamera2 python3-jinja2
python3-yaml python3-ply
sudo apt install -y libboost-dev libgnutls28-dev openssl
libtiff5-dev pybind11-dev qtbase5-dev libqt5core5a libqt5gui5
libqt5widgets5 libglib2.0-dev libgstreamer-plugins-base1.0-dev
```

```
libboost-program-options-dev libdrm-dev libexif-dev libpng-dev
nlohmann-json3-dev
sudo apt install -y meson cmake ninja-build
```

b. `libCamera` setup

i. Go to the `rpicam-apps` folder and clone the `libcamera` library. Make sure to check out the branch to commit number `29156679`.

```
cd /home/pi/{source/sink}_code/rpicam-apps
git clone git@github.com:raspberrypi/libcamera.git
cd libcamera
git checkout 29156679
```

ii. Run the following commands to build and install the library.

```
meson setup build --buildtype=release
-Dpipelines=rpi/vc4,rpi/pisp -Dipas=rpi/vc4,rpi/pisp
-Dv4l2=true -Dgstreamer=enabled -Dtest=false
-Dlc-compliance=disabled -Dcam=disabled -Dqcam=disabled
-Ddocumentation=disabled -Dpycamera=enabled

ninja -C build
sudo ninja -C build install
```

c. `rpicam-apps` setup

Go to the rpicam-apps folder and run the following commands.

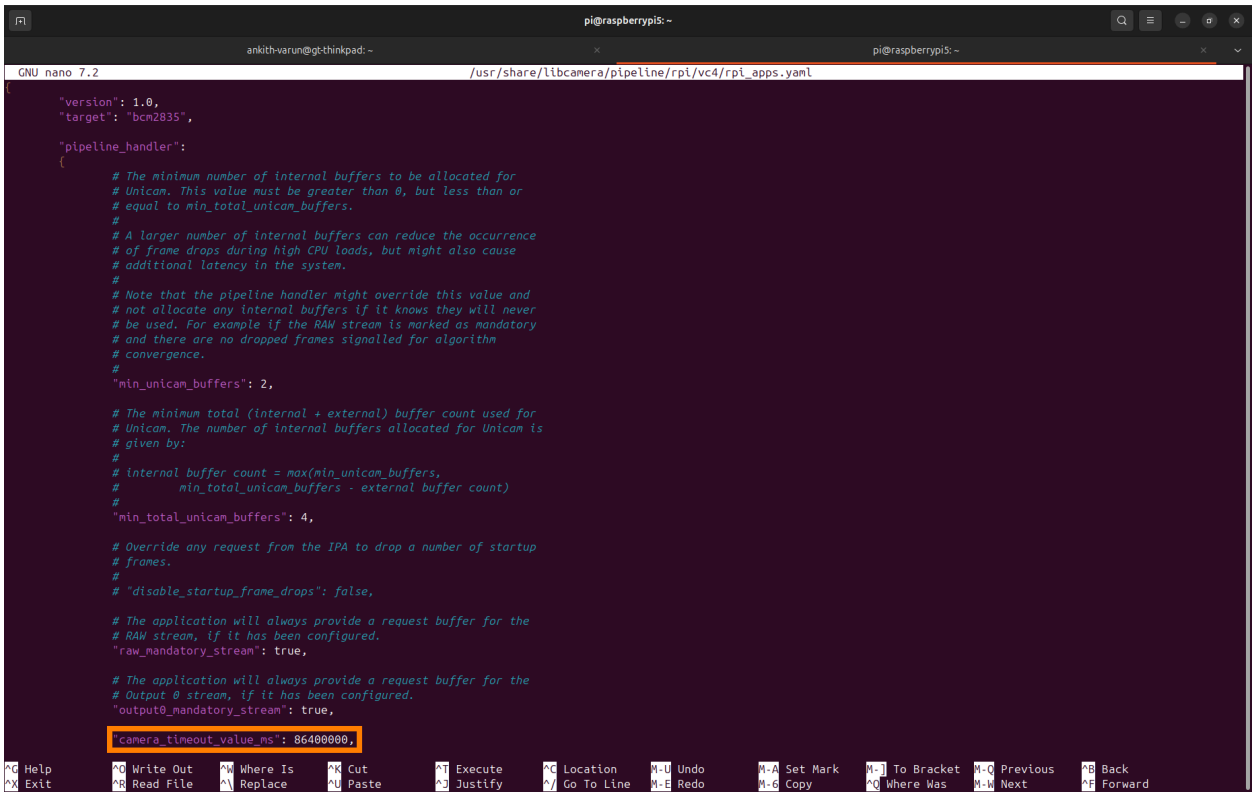
```
meson setup build -Denable_libav=disabled -Denable_drm=enabled
-Denable_egl=disabled -Denable_qt=disabled
-Denable_opencv=disabled -Denable_tflite=disabled
-Denable_hailo=disabled

meson compile -C build
sudo meson install -C build
sudo ldconfig
```

d. Sink camera configuration

For sink add camera timeout to make it wait for the source. Modify the `rpicam_apps.yaml` file.

```
sudo nano /usr/share/libcamera/pipeline/rpi/vc4/rpi_apps.yaml
```

Add `"camera_timeout_value_ms": 86400000` at the end of the dictionary and press Ctrl + S and Ctrl +X to save and close the file.



4. `NTP setup`

a. On all pis:

```
sudo apt update
sudo apt install -y ntp ntpdate
```

b. On Source pi:

```
sudo ufw allow 123/udp
```

5. Tmux logging setup

Copy the `.tmux` folder from the `new_{source/sink}` folders to the home directory. Use the following command.

```
cp -r /home/pi/{source/sink}_code/.tmux /home/pi/
```

This ensures all the tmux sessions print statements are logged in `~/.tmux/logs` folder.

6. Virtual Environment setup

Uploading the captured videos to AWS requires the `boto3` package which can only be installed using pip. So we create a virtual environment called `3d_vision` and install it inside it.

```
python -m venv /home/pi/3d_vision
source /home/pi/3d_vision/bin/activate
pip install boto3
```

COMMON ERRORS AND THEIR SOLUTIONS

Error Type 1 :-Failed to recognize camera

```
pi@raspberrypi5:~ $ rpicam-hello
[0:00:43.016913353] [888]  INFO Camera camera_manager.cpp:327 libcamera v0.4.0+53-29156679
Preview window unavailable
ERROR: *** no cameras available ***
```

Error 1

```
pi@raspberrypi5:~/sink_code/old_files $ python live_stream.py
[0:01:12.337711654] [911]  INFO Camera camera_manager.cpp:327 libcamera v0.4.0+53-29156679
Traceback (most recent call last):
  File "/home/pi/sink_code/old_files/live_stream.py", line 117, in <module>
    picam2 = Picamera2()
             ^^^^^^^^^^^
  File "/usr/lib/python3/dist-packages/picamera2/picamera2.py", line 257, in __init__
    camera_num = self.global_camera_info()[camera_num]['Num']
                 ~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^
IndexError: list index out of range
```

Error 1(alternative form)

```
pi@raspberrypi5:~/sink_code/old_files $ python live_stream.py
[0:03:59.383923531] [909]  INFO Camera camera_manager.cpp:327 libcamera v0.4.0+53-29156679
[0:03:59.409046249] [912]  WARN RPiSdn sdn.cpp:40 Using legacy SDN tuning - please consider moving SDN inside rpi.denoise
[0:03:59.410753270] [912]  INFO RPI vc4.cpp:447 Registered camera /base/soc/i2c0mux/i2c@1/imx296@1a to Unicam device /dev/media4 and ISP device /dev/media1
[0:03:59.410827454] [912]  INFO RPI pipeline_base.cpp:1121 Using configuration file '/usr/share/libcamera/pipeline/rpi/vc4/rpi_apps.yaml'
[0:03:59.417018801] [909]  INFO Camera camera.cpp:1202 configuring streams: (0) 1280x720-XBGR8888 (1) 1456x1088-SBGGR10_CSI2P
[0:03:59.417446686] [912]  INFO RPI vc4.cpp:622 Sensor: /base/soc/i2c0mux/i2c@1/imx296@1a - Selected sensor format: 1456x1088-SBGGR10_1X10 - Selected unicam format: 1456x1088-pBAA
[0:03:59.973134599] [912] ERROR V4L2 v4l2_videodevice.cpp:2049 /dev/video0[15:cap]: Failed to start streaming: Input/output error
Traceback (most recent call last):
  File "/home/pi/sink_code/old_files/live_stream.py", line 120, in <module>
    picam2.start_recording(JpegEncoder(), FileOutput(output))
  File "/usr/lib/python3/dist-packages/picamera2/picamera2.py", line 1827, in start_recording
    self.start()
  File "/usr/lib/python3/dist-packages/picamera2/picamera2.py", line 1185, in start
    self.start_()
  File "/usr/lib/python3/dist-packages/picamera2/picamera2.py", line 1153, in start_
    self.camera.start(controls)
RuntimeError: Failed to start camera: Input/output error
```

Error 2

Possible reasons:-

- **The camera cable** is **loose**.During adjustment the camera cable might get pulled out a little. Re-attach the wire and confirm if the issue resolves.
- MAS pin is wrongly connected to some other voltage.make sure that we connect the MAS pin to the **1.65 V** voltage or remove it entirely.Anything other than this will cause the issue.
- Source live stream or captures should not be on.**You should always run the sink live_stream first and then of the source**.For **ERROR 1**, Restart both the pis to solve this issue.For **ERROR 2**,just stopping the source will work,you **don't** need to restart the pis again.

Note 1:-if nothing works,just restart the pis properly and then start sink live-stream first and then source. If this doesn’t work the best this is to replace the connections altogether.

Note 2:-XVS and XHS have no influence on the detection of the camera,don’t waste your time debugging them.